

Technical Design Review

Required Corrections Before Implementation

Review of Ahmed's Development Plan and Technical Design

Review status	Conditionally approved
Next action	Revise the design before implementation
Implementation	Do not begin until the corrections below are resolved

The design shows strong engineering thinking. The purpose of this review is not to reduce that quality, but to make the V1 implementation correct, bounded, and demonstrable.

1. Review Outcome

Your plan demonstrates strong understanding of the problem. The generic Resource-based domain model is appropriate, the concurrency section is thoughtful, and the decision to keep PostgreSQL as the source of truth is correct. The plan also shows good awareness of testing, tenant isolation, cache invalidation, idempotency, and local reproducibility.

Status: The plan is conditionally approved. Before implementation begins, revise the design to address the mandatory corrections in this document. The goal is to keep the architecture strong while preventing unnecessary V1 complexity.

The strongest parts of the current plan include:

- A generic booking domain based on Business, Location, Service, Resource, Availability, and Booking rather than hardcoded business-specific logic.
- Database-level protection against overlapping bookings using PostgreSQL range constraints, supported by real concurrency tests.
- Server-authoritative booking duration and booking snapshots for historical consistency.
- Redis treated as a performance layer rather than a source of truth.
- Clear booking state transitions, authorization boundaries, and testable error behavior.
- A reproducible local topology using Docker Compose with PostgreSQL, Redis, API, and Worker.

However, V1 currently contains more architecture and operational machinery than is necessary for this assessment. There is also one important correctness issue in the slot-generation algorithm and one design decision around booking buffers that must be made explicit before code is written.

2. Mandatory Correction 1 - Fix the Slot Generation Algorithm

The current design defines a configurable slot granularity, but the pseudocode advances the cursor by service duration plus buffer after emitting a slot. That behavior does not use slot granularity as the candidate start-time interval.

Example

Working hours: 09:00 - 12:00
Service duration: 30 minutes
Slot granularity: 15 minutes
Buffer: 0 minutes

With 15-minute granularity, valid candidate starts should include 09:00, 09:15, 09:30, 09:45, 10:00, and so on. The current cursor progression would produce only 09:00, 09:30, 10:00, 10:30, etc. That removes valid booking choices.

Required change: Separate candidate-slot generation from occupancy checks. Advance the candidate start by `slotGranularityMinutes`. For every candidate, test the interval `[candidate, candidate + serviceDuration)` against working windows, breaks, existing bookings, and the buffer policy.

```
candidate = firstAlignedStart
while candidate + duration <= workingWindowEnd:
    if intervalsBookable(candidate, candidate + duration):
        emit candidate
    candidate = candidate + slotGranularityMinutes
```

The optimized implementation may jump over a known conflict for performance, but the behavior must remain equivalent to evaluating the granularity-aligned candidate starts. Add tests where service duration is different from slot granularity, including 45-minute services with 15-minute granularity.

3. Mandatory Correction 2 - Define the Buffer Integrity Model

The current PostgreSQL exclusion constraint protects the booking interval [startAt, endAt), while buffer enforcement is handled by application logic under an advisory lock. This can be a valid design, but the plan must explicitly decide what buffer means for correctness.

Example

```
Booking A: 10:00 - 10:30
Buffer after A: 15 minutes
Business rule: next booking must not start before 10:45
```

A database constraint protecting only [10:00, 10:30) does not itself reject a new booking starting at 10:35. The application can reject it, but then buffer separation is not a database-level invariant.

Choose and document one of these models:

Model	Meaning	Requirement
A. DB invariant	The buffer is part of the protected occupancy interval.	Persist or derive a protected range that includes the required buffer and enforce it at database level.
B. App policy	The database guarantees only physical booking overlap; buffer is a scheduling policy.	Enforce buffer consistently inside the locked transaction and state clearly that the DB exclusion constraint alone does not enforce buffer spacing.

Required change: Select one model and update the concurrency section, availability logic, booking transaction, and tests so they all use the same definition. Do not leave buffer semantics implicit.

4. Mandatory Correction 3 - Remove Automatic No-Show from V1

The plan currently allows the Worker to automatically move a confirmed booking to no_show after startAt + noShowAfterMinutes. That assumption is not reliable without a check-in mechanism or an external source that can prove whether the customer actually arrived.

For V1, keep no-show as an explicit human action:

confirmed -> no_show
Actor: Admin or authorized Staff
Condition: manual operational decision

Background jobs should be reserved for deterministic events. Examples that are appropriate for V1 include pending-booking expiration and reminder jobs. Those actions depend on known timestamps and system state, not on an unobserved real-world event.

Required change: Remove automatic no-show scheduling and processing from V1. Keep manual no-show as a valid state transition. You may document automatic no-show as a future enhancement if a reliable attendance/check-in signal is introduced.

5. Mandatory Correction 4 - Split Mandatory V1 from Advanced / Bonus Scope

The current document is technically ambitious. Several features are defensible in a production SaaS platform, but making all of them mandatory weakens the assessment because the project can become architecture-heavy before a complete working booking system exists.

Update the plan so every feature is classified as either Mandatory V1 or Advanced / Bonus. The first priority is a complete, correct, locally runnable system.

Recommended Mandatory V1

- Node.js + TypeScript + Fastify.
- PostgreSQL + Prisma with migrations and seed data.
- Redis availability cache with TTL and correct invalidation behavior.
- BullMQ Worker for at least pending-booking expiration and one demonstration reminder job.
- Admin and Customer authentication with JWT and a practical refresh/logout flow.
- Business, Location, Service, Resource, ServiceResource, AvailabilityRule, AvailabilityException, and Booking.
- Booking lifecycle, cancellation rules, manual no-show, and completion rules.
- Database-backed double-booking protection and real concurrency tests.
- POST /bookings idempotency.
- Audit events for important administrative/booking actions.
- Swagger/OpenAPI, structured error responses, Docker Compose, README, and unit/integration/API/concurrency tests.

Recommended Advanced / Bonus

- Staff invitation lifecycle and granular per-business permission administration.
- Refresh-token families, reuse detection, and more advanced session security beyond the practical V1 requirement.
- Transactional Outbox and a dedicated Outbox Publisher.
- Complex worker deduplication/recovery mechanisms beyond the required idempotent jobs.
- A full DDD/Hexagonal package structure where it adds significant implementation overhead.
- Extra infrastructure sophistication that is not required to run and prove the booking engine locally.

Important: Advanced items are not rejected. They are intentionally deferred so the assessment first proves that you can deliver a complete working system. If Mandatory V1 is stable and well tested, advanced items can be added afterward.

6. Mandatory Correction 5 - Resolve Booking Domain Ownership Before Coding

The current folder design shows booking logic under `apps/api/modules/bookings` and also introduces `packages/booking-core` so the API and Worker can share the same state machine/use cases. The direction toward a shared booking core is reasonable, but the plan must settle on one authoritative home for the booking domain before implementation starts.

Required rule:

- One booking domain model
- One state machine
- One set of booking policies
- One transition implementation
- Consumed by API and Worker
- No duplicated business logic

You may keep HTTP presentation adapters inside `apps/api` and job adapters inside `apps/worker`. The domain rules and shared transition behavior must not be independently implemented in both applications.

7. Required Revised Submission

Before writing production implementation code, submit a revised version of the technical design. You do not need to rewrite the entire document. Update the affected sections and provide a short change summary.

#	Deliverable	Expected content
1	Corrected slot-generation design	Show the candidate-start algorithm, granularity behavior, and the new mandatory test cases.
2	Explicit buffer decision	State whether buffer is a DB invariant or application scheduling policy, then align constraints, transaction logic, and tests.
3	Manual no-show in V1	Remove automatic no-show worker behavior from V1 and keep the transition manual.
4	Scope matrix	Clearly separate Mandatory V1 from Advanced / Bonus.
5	Booking ownership	Choose the single authoritative location for booking domain rules used by both API and Worker.

6	Updated phase plan	Revise the implementation phases to reflect the smaller V1 and defer bonus work until the mandatory system is complete.
7	Change summary	List exactly what changed from the current design and why.

Implementation gate: Do not start the full implementation until the revised design is reviewed. A small technical spike for the PostgreSQL overlap/concurrency approach is acceptable, but the production project structure and booking engine should follow the corrected plan.

8. What Will Be Evaluated During Implementation

The final assessment will be based primarily on working engineering evidence, not on the size of the architecture document.

- Can the full system start locally from a clean clone using the documented Docker flow?
- Can the same backend model multiple business types without business-specific booking code?
- Does availability produce correct slots across durations, granularity, exceptions, timezones, and buffers?
- Can the system prove that double booking is impossible under the tested concurrency model?
- Does idempotency prevent duplicate booking creation during client retries?
- Are authorization and tenant/business boundaries enforced?
- Do Redis and BullMQ add useful behavior without becoming sources of truth?
- Are failures expressed through stable API errors rather than raw exceptions?
- Do unit, integration, API, and concurrency tests prove the key claims?
- Is the code understandable, maintainable, and proportional to the problem?

A smaller architecture that is correct, complete, tested, and easy to explain will score higher than a large architecture with unfinished or unproven subsystems.

9. Final Note

The amount of thought in your current plan is appreciated. In particular, the concurrency, database-integrity, and testing sections show good engineering direction. The requested corrections are intended to improve correctness and execution discipline, not to remove that depth.

Revise the design using the points above, keep V1 focused on a complete working booking platform, and preserve the advanced ideas as clearly separated follow-up work. Once the revised design is accepted, implementation can proceed phase by phase with review checkpoints.